

解 説

移動ロボットのための確率的な自己位置推定と地図構築

Probabilistic Approaches to Localization and Mapping for Mobile Robots

友 納 正 裕* *千葉工業大学

Masahiro Tomono* *Chiba Inst. of Tech.

1. は じ め に

自己位置推定と地図構築は、移動ロボットにとって最も基本的な機能である。自己位置推定は、自分の位置を確認しながら目的地に正しく向かうために、移動時のあらゆる場面で必要である。地図は自己位置推定や経路計画に必要であり、地図の構築や更新は多大な工数がかかるので、ロボットが自分で地図を作ることが望ましい。

自己位置推定や地図構築はセンサデータに基づいて行われる。センサデータには不確実性が伴うため、確率に基づく方法が用いられる。本稿では、そのなかで主流の一つをなすベイズフィルタによる自己位置推定および地図構築の方法を紹介する。また、他の方法として最適化アプローチにも言及する。

簡単のため、本稿では車輪型移動ロボットが水平面を走る場合を考える。センサはロボットに搭載されているとし、GPSは考えない。ロボットの位置は $(x, y, \theta)^T$ の3自由度で表す。 x, y は地図上の座標値、 θ はロボットの向きである。また、地図 m はランドマーク m_j の集合とし、 m_j の位置は二次元の点 $(m_{j,x}, m_{j,y})^T$ で表す。

2. 自己位置推定

自己位置推定とは、地図は与えられているとして、自分が地図上のどこにいるかをセンサデータから推定する問題である。自己位置推定での状態ベクトルは、時刻 t におけるロボットの位置 $r_t = (x_t, y_t, \theta_t)^T$ である。

自己位置推定には、位置追跡と大域自己位置推定がある。紙数の制限のため、本稿では位置追跡を説明する。

2.1 位置追跡

位置追跡 (position tracking) では、ロボット位置の初期値を外部から与え、その初期値を始点に前時刻の位置から現時刻の位置を逐次的に推定する。

確率的な位置追跡は、所与の地図 m のもとで、外界セ

ンサの計測データの時系列 $z_{1:t} = z_1, z_2, \dots, z_t$ とロボット動作の時系列 $a_{1:t} = a_1, a_2, \dots, a_t$ が得られたときのロボット位置 r_t の確率密度 $p(r_t|z_{1:t}, a_{1:t}, m)$ の推定問題として定式化される。これはベイズ定理とマルコフ性仮定を使って次のように変形できる [1]。

$$p(r_t|z_{1:t}, a_{1:t}, m) = \alpha p(z_t|r_t, m) \times \int p(r_t|r_{t-1}, a_t) p(r_{t-1}|z_{1:t-1}, a_{1:t-1}, m) dr_{t-1} \quad (1)$$

これは漸化式であり、時刻 $t-1$ のロボット位置の確率密度が分かれば時刻 t のロボット位置の確率密度が求まる。この漸化式は、後述のように、カルマンフィルタやパーティクルフィルタなどのベイズフィルタにより計算される。

ここで重要なのは、状態ベクトル r_t は直接観測できず、観測可能なデータ $z_{1:t}$ と動作 $a_{1:t}$ から推定することである。 $p(r_t|r_{t-1}, a_t)$ と $p(z_t|r_t, m)$ が、それを担う。前者を動作モデル (motion model)、後者を計測モデル (measurement model) という。まず、これらを説明する。

2.2 動作モデル

動作モデル $p(r_t|r_{t-1}, a_t)$ は、時刻 $t-1$ のロボット位置が r_{t-1} で動作コマンドが a_t のとき、時刻 t でロボット位置が r_t となる確率密度である。ここで不確実性をもたらすのは a_t による動作であり、 r_{t-1} は固定である。 r_{t-1} も誤差をもつが、それは式 (1) の積分によって反映される。

いま、ロボットの速度を v_t 、角速度を ω_t として、 $a_t = (v_t, \omega_t)^T$ とする。そして、動作コマンドによる指令値をあらためて $\hat{a}_t = (\hat{v}_t, \hat{\omega}_t)^T$ と書く (文献 [1] の記法と逆なので注意)。 a_t は真値であり、実際に観測できるのは $\hat{a}_t$ である。実装においては、オドメトリによる計測値を $\hat{a}_t$ とすることも多い。 ε_t を誤差とすると、 $a_t = \hat{a}_t + \varepsilon_t$ と表せる。なお、この書き換えにより動作モデルは $p(r_t|r_{t-1}, \hat{a}_t)$ となる。この a_t を用いて、ロボット位置 r_t は式 (2) のように計算できる [1] [2]。 Δt は $t-1 \sim t$ 間の微小時間である。

$$g(r_{t-1}, a_t) = \begin{pmatrix} x_{t-1} \\ y_{t-1} \\ \theta_{t-1} \end{pmatrix} + \begin{pmatrix} v_t \cos \theta_{t-1} \\ v_t \sin \theta_{t-1} \\ \omega_t \end{pmatrix} \Delta t \quad (2)$$

このように微小変位を累積して位置推定を行うことがで

原稿受付 2011 年 3 月 31 日

キーワード: Localization, Mapping, SLAM, Bayes Filter, Mobile Robot

*〒 275-0016 習志野市津田沼 2-17-1

*Narashino-shi, Chiba

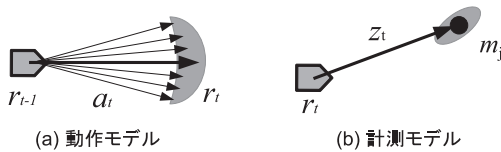


図 1 動作や計測には誤差が伴う。灰色部が誤差分布

きるが、この方法は誤差が累積するという問題がある。人間に例えると、目をつぶって歩数で距離や位置を推測するようなものである。このため、長距離を移動すると誤差が大きくなり、計測モデルによる位置修正が必要になる。

2.3 計測モデル

計測モデル $p(z_t|r_t, m)$ は、地図 m でロボット位置が r_t のとき、計測データ z_t が得られる確率密度である。 z_t はセンサ座標系での値をとる。ただし、計測データからロボット位置を推定する場合、 z_t は定数で、 r_t が変数であり、 $p(z_t|r_t, m)$ は r_t の尤度 (likelihood) とよばれる。

計測モデルを厳密に書くと、ランドマークと計測データの対応づけ関数を c_t として、 $p(z_t|r_t, c_t, m)$ となる。データ対応づけ (data association) はそれ自体が大きな技術課題であり、ここでは簡単のため、 c_t は所与とする。

計測モデルはセンサに大きく依存する。本稿では、レーザスキャナの例を示す。レーザスキャナでは対象物までの距離と方向が得られる。いま、図 1 (b) で、ランドマーク m_j の位置 $(m_{j,x}, m_{j,y})^T$ が地図に登録されているとすると、ロボット位置 r_t で得る計測データは次のようになる。

$$h(r_t, m_j) = \begin{pmatrix} \sqrt{(m_{j,x} - x_t)^2 + (m_{j,y} - y_t)^2} \\ \tan^{-1}(\frac{m_{j,y} - y_t}{m_{j,x} - x_t}) - \theta_t \end{pmatrix} \quad (3)$$

m_j の実測値 z_t に対して $z_t = h(r_t, m_j)$ という制約関係が成り立つ。しかし、ランドマーク 1 個では、この関係から r_t を一意に求めることはできない。また、良いランドマークが存在しない場所もあり、観測は不連続になりうる。これらの問題は動作モデルで補うことができる。

2.4 カルマンフィルタによる位置追跡

動作モデルと計測モデルをベイズフィルタで融合することで、上で述べたそれぞれの短所が補完される。カルマンフィルタを用いる方法では、ロボット位置、動作モデル、計測モデルをそれぞれ一つの正規分布で表す。

動作モデルを式 (4) に示す。 g は非線形関数なので、拡張カルマンフィルタ (EKF) で扱うために、テーラー展開で線形化する。 G_t は r_t に関する g のヤコビ行列、 V_t は a_t に関する g のヤコビ行列である。 $\hat{a}_t$ を動作指令値あるいはオドメトリ値とし、 a_t は正規分布 $N(\hat{a}_t, M_t)$ に従うとする。

$$\begin{aligned} r_t &= g(r_{t-1}, a_t) \\ &\approx g(\bar{r}_{t-1}, \hat{a}_t) + G_t(r_{t-1} - \bar{r}_{t-1}) + V_t(a_t - \hat{a}_t) \end{aligned} \quad (4)$$

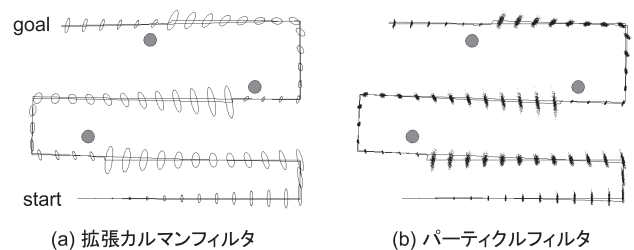


図 2 自己位置推定のシミュレーション。ロボット軌跡は真値と推定値を表示。3 個の丸がランドマーク、楕円または点群がロボット位置の分布。(a) と (b) で地図は同じだが、乱数を用いているのでロボット軌跡はまったく同じではない

計測モデルを式 (5) に示す。 h も非線形関数なので、テーラー展開で線形化する。 H_t は r_t に関する h のヤコビ行列である。 β_t は誤差で、正規分布 $N(0, Q_t)$ に従うとする。

$$\begin{aligned} z_t &= h(r_t, m_j) + \beta_t \\ &\approx h(\bar{r}_t, m_j) + H_t(r_t - \bar{r}_t) + \beta_t \end{aligned} \quad (5)$$

ロボット位置の分布を $N(\bar{r}_t, \Sigma_t)$ とすると、拡張カルマンフィルタにより次のように計算できる [1] [2]。

(1) 動作モデルによる r_t の予測

$$\bar{r}_t = g(\bar{r}_{t-1}, \hat{a}_t) \quad (6)$$

$$\tilde{\Sigma}_t = G_t \Sigma_{t-1} G_t^T + V_t M_t V_t^T \quad (7)$$

(2) 計測モデルによる r_t の更新

$$K_t = \tilde{\Sigma}_t H_t^T (H_t \tilde{\Sigma}_t H_t^T + Q_t)^{-1} \quad (8)$$

$$\bar{r}_t = \bar{r}_t + K_t(z_t - h(\bar{r}_t, m_j)) \quad (9)$$

$$\Sigma_t = (I - K_t H_t) \tilde{\Sigma}_t \quad (10)$$

カルマンフィルタの解は、動作モデルの値 $\bar{r}_t$ と、実測値 z_t と計測モデルの値 $h(\bar{r}_t, m_j)$ の差とを K_t で重みづけした線形和になっている。 K_t はカルマンゲインとよばれ、動作モデルと計測モデルの精度に応じた重みをつけて最適値を得る働きをしている。また、カルマンゲインは、 z_t をセンサ座標系から地図座標系に変換する働きもしている。このように、カルマンフィルタにより、複数のデータを融合して最適解を得ることができる。

2.5 パーティクルフィルタによる位置追跡

パーティクルフィルタによる方法では、多数のパーティクルでロボット位置の確率分布を表す。 i 番目のパーティクル $p_t^{(i)}$ は、ロボット位置 $r_t^{(i)} = (x_t^{(i)}, y_t^{(i)}, \theta_t^{(i)})$ と重み $w_t^{(i)}$ をもつ。位置追跡の手順を以下に示す [1]。

(1) 動作モデルによる r_t の予測

各パーティクルを $\hat{a}_t$ だけ進める。動作の誤差分布 $N(0, M_t)$ に従って乱数 $\varepsilon_t^{(i)}$ を発生させ、誤差として $\hat{a}_t$ に加える。

$$r_t^{(i)} = g(r_{t-1}^{(i)}, \hat{a}_t + \varepsilon_t^{(i)}) \quad (11)$$

(2) 計測モデルによる重み計算

各パーティクルから見えるランドマークの位置とセンサデータとの一致度から尤度を計算し、重みとする。重みは合計が1になるように正規化する。

$$w_t^{(i)} \propto p(z_t | r_t^{(i)}, m_j) \quad (12)$$

(3) リサンプリング

各 $p_t^{(i)}$ に対し、 $w_t^{(i)}$ に比例した個数だけ複製を作り、時刻 $t+1$ のパーティクルとする。

パーティクルフィルタによる自己位置推定では、各パーティクルがロボットを模擬していると考えればよい。動作モデルに従ってパーティクルが移動し、その位置から見えるべきランドマークが予測と実測でよく一致すれば重みが大きくなる。重みの大きなパーティクルはリサンプリングによって多くの複製が作られ、その周辺の密度が高くなる。このパーティクルの密度でロボット位置の分布を表す。

パーティクルフィルタの利点は、正規分布に限らずに多様な確率分布を扱えることである。短所は、状態ベクトルの次元が大きくなると、必要なパーティクル数が増大し、多大な計算時間がかかることである。

3. 地図構築

地図構築は自己位置推定と分離しては実行できず、SLAM (Simultaneous Localization and Mapping) という枠組みが提案された。

3.1 SLAM

センサはロボットに載っているため、その計測データはロボットからの相対値で得られる。そこで、計測データをロボット位置で座標変換し、地図座標系でのランドマーク位置を求めれば地図が構築できる。しかし、前述のように、動作モデルだけで自己位置推定を行うと累積誤差が生じるため、ランドマークの計測による修正が必要である。ここで問題なのは、ランドマークを載せた地図はこれから作るものであり、今は存在しないことである。したがって、ロボット位置と地図は同時に推定しなければならないことになる。これがSLAMである。

SLAMを解くポイントは、同じランドマークを複数回観測することである。基本的には、それまでに観測したランドマークを使ってロボット位置を推定し、そのロボット位置に従って新たなランドマークを配置して地図を成長させる、というプロセスを繰り返せばよい。

SLAMにもベイズフィルタが使われる。SLAMは、計測データ $z_{1:t}$ と動作 $a_{1:t}$ のもとでの、ロボット位置 r_t (または軌跡 $r_{1:t}$) と地図 m の同時確率密度の推定問題として定式化される [1]。状態ベクトルは、ロボット位置 r_t (または軌跡 $r_{1:t}$) とランドマーク集合 $m_1, \dots, m_n$ からなるので、その次元は一般に非常に大きくなる。

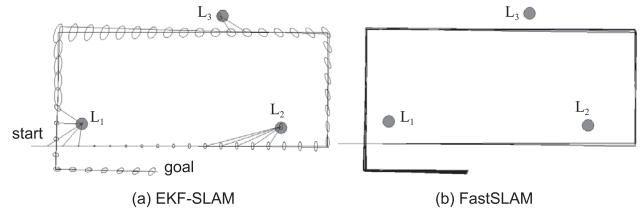


図3 SLAMのシミュレーション。ロボットの軌跡からランドマークへの線分は観測を表す。(a)では周回後に再びランドマーク L_1 を観測することで、ロボット位置の不確実性が減少している。(b)も同様だが、FastSLAMは軌跡単位でフィルタリングするので、良い軌跡だけ残される

3.2 カルマンフィルタを用いたSLAM

拡張カルマンフィルタを用いたSLAM (EKF-SLAM) は、最も歴史の古い方法である [3]。ロボット位置 r_t と地図 m の同時確率密度を正規分布で表し、自己位置推定と同様に漸化式を構成する。

$$\begin{aligned} p(r_t, m | z_{1:t}, a_{1:t}) &= \alpha p(z_t | r_t, m) \\ &\times \int p(r_t | r_{t-1}, a_t) p(r_{t-1}, m | z_{1:t-1}, a_{1:t-1}) dr_{t-1} \end{aligned} \quad (13)$$

いま、 $u_t = (r_t, m_1, \dots, m_n)$ とすると、求める同時確率は $p(u_t | z_{1:t}, a_{1:t})$ となり、自己位置推定と同様に拡張カルマンフィルタで実装できる。ただし、動作モデルは自己位置推定と同様に r_t に対して計算するが、計測モデルは u_t に対して計算する。 u_t は次元が大きいため、計算量は増える。

3.3 パーティクルフィルタを用いたSLAM

地図構築では状態ベクトルの次元が非常に大きくなるため、パーティクルフィルタをそのまま適用するのは難しい。そこで、Rao-Blackwellizationという操作により、パーティクルのサンプル空間をロボット軌跡に限定して次元を減らし、ランドマーク位置の分布はパーティクルごとに解析的に計算するという方法が提案された [4]。これは、Rao-Blackwellized particle filter (RBPF) SLAMとよばれる。その実用的な実装としてFastSLAMがある [1]。FastSLAMでは、ロボット軌跡 $r_{1:t}$ と地図 m の同時確率密度を式 (14) のように分解し、ロボット軌跡の分布 $p(r_{1:t} | z_{1:t}, a_{1:t})$ をパーティクルフィルタで、ランドマーク位置の分布 $p(m_j | r_{1:t}, z_{1:t})$ をカルマンフィルタで計算する。各パーティクルはロボット軌跡 $r_{1:t}^{(i)}$ 、地図 m 、重み $w_t^{(i)}$ をもつ。

$$\begin{aligned} p(r_{1:t}, m | z_{1:t}, a_{1:t}) \\ = p(r_{1:t} | z_{1:t}, a_{1:t}) \prod_j p(m_j | r_{1:t}, z_{1:t}) \end{aligned} \quad (14)$$

RBPF-SLAMでは、パーティクルフィルタで表す状態ベクトルをロボット軌跡に限定するが、その次元はロボット位置に比べればまだ大きい。このため、大量のパーティクルが必要になるように思えるが、一つの軌跡中の各ロボット

位置は強い相関をもつので、短時間ならば必要なパーティクル数は多くない。しかし、長時間実行するとFastSLAMは誤差分散を小さく見積りすぎるといった報告がある[5]。また、誤差分布の形状やデータ対応づけの精度によっては、仮説が多数分岐してパーティクル数が足りなくなる可能性もある。

4. フィルタアプローチと最適化アプローチ

自己位置推定と地図構築へのベイズフィルタによるアプローチを紹介したが、他の有力な方法として最適化アプローチがある。これらのアプローチについて、筆者の経験も踏まえて見解を述べる。

自己位置推定のように状態ベクトルの次元が小さい場合は、ベイズフィルタは実用性が高く非常に有効である。ロボット位置を、誤差分散も含めて、走行しながら逐次的に推定できることは、ロボットの移動を伴うタスクにおいて極めて有用である。

一方、地図構築は状態ベクトルの次元が膨大（数千～数百万）になりうるため、ベイズフィルタの適用は困難が伴う。EKF-SLAMの計算量は、次元数の二乗に比例するため、ランドマークが多い地図には向かない。RBPF-SLAMも前述の問題がある。

最適化アプローチによる地図構築では、動作モデルと計測モデルから評価関数を構成し、その最適解を求めることにより、ロボット位置とランドマーク位置を同時に求める。ランドマークが多ければ、動作モデルは省略できる。手法として、スキャンマッチング[6]、Structure from Motion[7]、グラフベースSLAM[8]、バンドル調整[9]などがある。利点は、大量のランドマーク（点群）を扱えることである。一般に、有意なランドマーク数が増えるほど、推定精度はよくなりロバスト性も高くなる。また、拡張カルマンフィルタでは線形化による誤差が生じるが、最適化アプローチでは非線形最適化を直接適用できるため精度が高い。単眼カメラによるSLAMでは、同じ計算量ならバンドル調整の方がベイズフィルタよりも精度が良いという報告がある[10]。

最適化アプローチでは誤差分散の正確な計算が困難な場合があるが、静的環境の地図であれば、誤差分散なしでも推定精度の高いほうが望まれることも多いであろう。また、ランドマークが多いと自己位置推定にも有利である。このため、筆者自身は、地図構築に最適化アプローチをとることが多い。なお、最適化アプローチの多くは最尤推定に基づき、確率理論を背景にもつ。また、フィルタアプローチとの併用もありうる。

本稿では割愛したが、データ対応づけは極めて重要である。データの誤対応（アウトライア）は致命的な結果をもたらすことが多く、適切な対処が必要である。SLAMでは、主に位置関係に基づいて最大尤度のランドマークを選択してデータ対応づけを行うことが多い[1]。一方、SIFT記述子[11]などの特徴量でランドマークを識別すると、精度の

高いデータ対応づけが可能になる。また、RANSAC[12]やM推定[13]などのロバスト推定もアウトライアに有効である。こうした様々な手法により、ロバストで高精度な自己位置推定や地図構築が可能になる。

5. ま と め

移動ロボットの確率的な自己位置推定と地図構築の方法として、ベイズフィルタによるアプローチを紹介した。ベイズフィルタの利点は、データ融合を統一的に行えること、ロボット位置などの不確実性を表せることである。また、最適化アプローチも有力な方法であり、フィルタアプローチとの使い分けや併用は興味深い課題である。

参 考 文 献

- [1] S. Thrun, W. Burgard and D. Fox: Probabilistic Robotics. MIT Press, 2005. (邦訳) 上田隆一訳: 確率ロボティクス, 毎日コミュニケーションズ, 2007.
- [2] 坪内孝司: ‘移動体の位置認識’, 計測自動制御学会編, ビークル, コロナ社, 2003.
- [3] R. Smith and P. Cheeseman: “On the Representation and Estimation of Spatial Uncertainty,” International Journal of Robotics Research, vol.5, no.4, pp.56–68, 1986.
- [4] K. Murphy and S. Russell: ‘Rao-Blackwellised Particle Filtering for Dynamic Bayesian Networks,’ A. Doucet ed.: Sequential Monte Carlo Methods in Practice. Springer, 2001.
- [5] T. Bailey, J. Nieto and E. Nebot: “Consistency of the Fast-SLAM Algorithm,” Proceedings of International Conference on Robotics and Automation, pp.424–429, 2006.
- [6] P.J. Besl and N.D. McKay: “A Method of Registration of 3-D Shapes,” IEEE Transactions on Pattern Analysis and Machine Intelligence, vol.14, no.2, pp.239–256, 1992.
- [7] R. Hartley and A. Zisserman: Multiple View Geometry in Computer Vision. Cambridge University Press, 2000.
- [8] G. Grisetti, C. Stachniss, S. Grzonka and W. Burgard: “A Tree Parameterization of Efficiently Computing Maximum Likelihood Maps using Gradient Descent,” Proceedings of Robotics: Science and Systems, 2007.
- [9] B. Triggs, P.F. McLauchlan, R.I. Hartley and A.W. Fitzgibbon: “Bundle Adjustment – A Modern Synthesis,” Vision Algorithms99, LNCS 1883, pp.298–372, 2000.
- [10] H. Strasdat, J.M.M. Montiel and A.J. Davison: “Real-time Monocular SLAM: Why Filter?,” Proceedings of International Conference on Robotics and Automation, pp.2657–2664, 2010.
- [11] D.G. Lowe: “Distinctive Image Features from Scale-Invariant Keypoints,” International Journal of Computer Vision, vol.60, no.2, pp.91–110, 2004.
- [12] M. Fischler and R. Bolles: “Random Sample Consensus: a Paradigm for Model Fitting with Application to Image Analysis and Automated Cartography,” Communications ACM, vol.24, pp.381–395, 1981.
- [13] P.J. Huber: Robust Statistics. John Wiley and Sons, 1981.



友納正裕 (Masahiro Tomono)

1985年東京大学大学院修士課程修了。1985～1999年NEC。2002年筑波大学大学院博士課程修了。2002年～2006年科学技術振興機構さがけ研究者。2006年～2008年東洋大学。2008年より千葉工業大学未来ロボット技術研究センター。(日本ロボット学会正会員)